Sistema de biblioteca: exercício para casa

Algoritmos II · individual · entrega na **quinta-feira, 12/11, até 23h50**, pelo canal combinado

O que você vai fazer

Você vai montar, **função por função**, o miolo de um sistema de biblioteca: **cadastrar, buscar, alterar e remover livros**, **emprestar e devolver** com controle de quem está com o quê, e **três relatórios** simples. Os dados ficam em dicionários, listas e uma matriz, que é o que já estudamos; a novidade é que cada tarefa agora vira uma **função com parâmetros e return**.

Não há menu nem digitação: o sistema é feito de funções que recebem os dados por parâmetro e devolvem o resultado. Você escreve o **corpo** de 14 funções (a última é desafio) dentro de **um único arquivo**, o `solucao.py`, e o `testes.py` confere cada uma. **Reserve cerca de 2h30.**

Parte	Funções	O que você pratica
1 · Cadastro de livros	criar_livro · cadastrar_livro · buscar_livro · atualizar_livro · listar_titulos	dicionário de dicionários: criar, ler, alterar; return de dicionário, True/False e None
2 · Empréstimos e remoção	livros_do_aluno · livro_esta_emprestado · emprestar_livro · devolver_livro · remover_livro	dicionário de listas; regras em ordem; função chamando outra função; devolver dois valores
3 · Relatórios	relatorio_por_autor · relatorio_indisponiveis · relatorio_emprestimos_mes	contagem em dicionário; filtro; matriz (soma de linha e de coluna)
4 · Desafio	processar_movimentos	juntar tudo: um laço que chama as funções das partes anteriores

As funções ficam **da mais simples para a mais difícil**. Cada função é testada separadamente: se travar numa, siga para a próxima e volte depois. (Só as partes 2 e 4 reaproveitam funções anteriores, e a ficha avisa quando for o caso.)

Como trabalhar

- Baixe `solucao.py` e `testes.py` e deixe os dois na **mesma pasta**. Escreva o código **só no solucao.py**, no lugar de `pass`. **Não mude o nome das funções nem os parâmetros**; o teste chama cada função pelo nome.
- Para conferir: `python testes.py` no terminal. Para cada função ele mostra [OK] ou [X]; quando falha, mostra **o caso que falhou, os dados de entrada, o que a sua função devolveu, o que era esperado** e uma dica. Rode quantas vezes quiser.
- O teste usa os exemplos deste texto e **também outros dados**. Por isso nada de valores fixos dentro das funções, e nada de variáveis globais: **use os parâmetros**.
- Não use `import` nem `input()`. As funções **devolvem** o resultado (`return`); elas não imprimem. Para espiar o que está acontecendo, escreva `print(...)` no fim do arquivo, na área de teste à mão, e rode `python solucao.py`. Pode deixar os prints no arquivo.
- No final do arquivo há **duas perguntas** (R1 e R2) para responder com as suas palavras. Elas fazem parte da entrega.

Funções em 5 minutos

Uma função é uma **receita com nome**: ela recebe ingredientes (**parâmetros**), faz o trabalho e entrega o prato pronto (**return**).

```
def calcular_media(nota1, nota2):      # def + nome + parâmetros (as entradas)
    soma = nota1 + nota2               # 'soma' só existe aqui dentro (variável local)
    return soma / 2                    # return: o que a função devolve a quem chamou

resultado = calcular_media(7, 9)       # chamada: 7 e 9 viram nota1 e nota2
print(resultado)                       # mostra 8.0
```

Ideia	Em palavras	Exemplo
return ≠ print	**return** entrega o valor a quem chamou a função (e o teste só enxerga isso). **print** só mostra na tela; sem return, a função devolve None.	`return soma / 2` (certo) `print(soma / 2)` (o teste recebe None)
devolver dois valores	Escreva dois valores separados por vírgula no return; quem chamou recebe os dois de uma vez, em duas variáveis.	`def dividir(a, b): return a // b, a % b` `quociente, resto = dividir(17, 5)` `# quociente = 3, resto = 2`
variável local	O que você cria dentro da função some quando ela termina. Se precisa de algo de fora, **receba por parâmetro**.	`soma` (do exemplo acima) não existe fora de `calcular_media`
função chama função	Uma função pode usar outra já escrita, como qualquer comando. Isso evita repetir código.	`lista = livros_do_aluno(emprestimos, aluno)`

Atenção: dicionários e listas que chegam por parâmetro

Um dicionário (ou lista) recebido como parâmetro é **o mesmo** que está fora da função, não uma cópia. Por isso comandos que **alteram** o objeto (`d[chave] = valor`, `lista.append(x)`, `lista.remove(x)`, `del d[chave]`) mudam o original, e a função **não precisa devolver** o dicionário. Já escrever `lista = ...` ou `d = {}` dentro da função cria uma **variável nova, local**, e o original não muda.

Certo: altera o original	Errado: cria uma variável local
`def adicionar(lista, x): lista.append(x)`	`def adicionar(lista, x): lista = lista + [x]`
`notas = [7]` `adicionar(notas, 9)` `# notas agora é [7, 9]`	`notas = [7]` `adicionar(notas, 9)` `# notas continua [7]`

Receita para cada função: (1) leia a ficha; (2) refaça um exemplo à mão, no papel; (3) escreva o corpo; (4) chame a função no fim do arquivo com os dados de exemplo e dê `print` no resultado; (5) rode `python testes.py` e leia a mensagem. Se uma função travar por mais de 20 minutos, deixe um comentário dizendo onde parou e siga para a próxima: função entregue pela metade vale mais que função em branco.

Cartão de referência: dicionários e listas

Operação	Exemplo	O que faz
inserir / trocar valor	`d[chave] = valor`	cria a chave ou substitui o valor dela
testar existência	`chave in d`	True se a chave existe (olha as chaves, não os valores)
ler valor	`d[chave]`	devolve o valor; **dá KeyError** se a chave não existe
ler com padrão	`d.get(chave, 0)`	devolve o valor, ou 0 se a chave não existe (sem erro)
remover chave	`del d[chave]`	tira a chave e o valor; KeyError se não existe
percorrer	`for chave in d:`	visita as chaves, na ordem em que foram inseridas
dicionário dentro de dicionário	`acervo[cod]["titulo"]`	primeiro acha o livro, depois o campo
lista	`l.append(x)` · `l.remove(x)` · `x in l`	remove tira a 1ª ocorrência; dá ValueError se x não está
tamanho	`len(l)` · `len(d)`	quantidade de itens (lista) ou de chaves (dicionário)
ler por posição	`t[0]` · `t[2]` · `m[i][j]`	a contagem começa em 0; m[i][j] = linha i, coluna j
percorrer por posição	`for i in range(len(l)):`	i vale 0, 1, 2, … até o último índice
acumulador	`soma = soma + x`	variável que começa em 0 e vai guardando o total
ainda sem valor	`melhor = None`	marca “ninguém ainda”; teste com melhor is None

Dois erros clássicos. (1) = guarda um valor e == compara dois valores. (2) Dentro de um `for` sobre um dicionário, a variável do laço recebe a *chave*, não o valor: para chegar no valor, use `d[chave]`.

Os dados do sistema

Todas as funções trabalham com estas três estruturas, que você também encontra, prontas, no começo do `solucao.py` (`acervo_exemplo`, `emprestimos_exemplo`, `matriz_exemplo`) para você testar à mão:

```
acervo = {                        # chave = código do livro; valor = dicionário do livro
  "L001": {"titulo": "Dom Casmurro", "autor": "Machado de Assis", "ano": 1899, "exemplares":
 2},
  "L002": {"titulo": "Capitães da Areia", "autor": "Jorge Amado", "ano": 1937, "exemplares":
 1},
  ...                             # "exemplares" = quantos estão DISPONÍVEIS agora
}
emprestimos = {"Ana": ["L001", "L004"], "Bruno": ["L002"]}   # aluno -> lista dos códigos qu
e ele tem
                                   # quem não tem livro nenhum NÃO aparece neste dicionário
matriz = [[10, 12, 9],             # cada linha é uma categoria (categorias = ["Romance", "Di
dático"])
          [ 4, 15, 8]]             # cada coluna é um mês (meses = ["Jan", "Fev", "Mar"])
```

Um aluno pode ficar com **no máximo 3 livros** ao mesmo tempo. Um livro só pode ser emprestado se tiver exemplar disponível. Todo empréstimo diminui os exemplares em 1 e toda devolução aumenta em 1.

Parte 1 — cadastro de livros · ≈ 45 min

Cinco funções para criar, guardar, procurar, alterar e listar livros. Cada exemplo abaixo parte dos dados originais de `solucao.py`.

1 · `criar_livro(titulo, autor, ano, exemplares)`	**dificuldade: fácil**
Recebe	quatro valores soltos: o título, o autor, o ano e a quantidade de exemplares.
Devolve	um dicionário de **um livro**, com as chaves "titulo", "autor", "ano" e "exemplares".
Regras	• Só monta e devolve o dicionário. Não precisa de `if` nem de laço. • Use `return`, não print: quem chamou precisa receber o dicionário.
Exemplos	`criar_livro("Dom Casmurro", "Machado de Assis", 1899, 2)` `  devolve {'titulo': 'Dom Casmurro', 'autor': 'Machado de Assis', 'ano': 1899, 'exemplares': 2}`
Pense assim	1) Escreva o dicionário com as quatro chaves, usando os parâmetros como valores. 2) Devolva-o.

2 · `cadastrar_livro(acervo, codigo, livro)`	**dificuldade: fácil**
Recebe	**acervo** (dicionário código → livro), **codigo** (texto) e **livro** (o dicionário que criar_livro devolve).
Devolve	**True** se cadastrou, **False** se recusou.
Regras	• Se o código **já é chave** do acervo: não grava nada e devolve False (o primeiro cadastro vale). • Se o código é novo: guarda o livro no acervo, com esse código como chave, e devolve True.

Exemplos	`novo = criar_livro("O Cortiço", "Aluísio Azevedo", 1890, 2)`
	`cadastrar_livro(acervo_exemplo, "L005", novo)`
	`  devolve True`
	`  depois: len(acervo_exemplo) vale 5`
	`  depois: acervo_exemplo["L005"]["titulo"] vale 'O Cortiço'`
	`novo = criar_livro("O Cortiço", "Aluísio Azevedo", 1890, 2)`
	`cadastrar_livro(acervo_exemplo, "L001", novo)`
	`  devolve False`
	`  depois: len(acervo_exemplo) vale 4`
	`  depois: acervo_exemplo["L001"]["titulo"] vale 'Dom Casmurro'`

Pense assim	1) Pergunte primeiro: este código já existe no acervo? (cartão: testar existência) 2) Se existe, devolva False agora, sem mexer em nada. 3) Se não existe, grave e devolva True.
Atenção	a função altera o acervo que veio de fora, então não devolve o acervo. Escrever `acervo = {...}` dentro dela criaria um acervo local e o de fora não mudaria.

3 · `buscar_livro(acervo, codigo)`	**dificuldade: fácil**
Recebe	**acervo** e o **codigo** procurado.
Devolve	o **dicionário do livro**, se o código existe; senão **None**.
Regras	• Não altera o acervo. • Código que não existe não pode dar erro: devolva None.

Exemplos	`buscar_livro(acervo_exemplo, "L003")`
	`  devolve {'titulo': 'Vidas Secas', 'autor': 'Graciliano Ramos', 'ano': 1938, 'exemplares': 0}`
	`buscar_livro(acervo_exemplo, "L999")`
	`  devolve None`

Pense assim	1) O código está no acervo? Se sim, devolva o livro dele. 2) Se não, devolva None (a palavra especial que significa “nada”).
Atenção	ler `acervo[codigo]` de um código que não existe dá KeyError. Teste antes.

4 · `atualizar_livro(acervo, codigo, campo, valor)`	dificuldade: média
Recebe	**acervo**, **codigo**, **campo** (o nome do campo, como "exemplares") e **valor** (o novo conteúdo).
Devolve	**True** se alterou, **False** se recusou.
Regras	• Devolve False, sem alterar nada, se o código não existe. • Devolve False, sem alterar nada, se o campo não é um dos campos do livro ("titulo", "autor", "ano", "exemplares"): o livro **não pode ganhar um campo novo**. • Senão, troca o valor do campo e devolve True.

Exemplos	`atualizar_livro(acervo_exemplo, "L003", "exemplares", 5)`
	`  devolve True`
	`  depois: acervo_exemplo["L003"]["exemplares"] vale 5`
	`atualizar_livro(acervo_exemplo, "L999", "ano", 2000)`
	`  devolve False`
	`atualizar_livro(acervo_exemplo, "L001", "editora", "X")`
	`  devolve False`
	`  depois: len(acervo_exemplo["L001"]) vale 4`

Pense assim	1) Primeira pergunta: o livro existe? 2) Segunda pergunta: o campo existe **dentro** do dicionário desse livro? 3) Só então troque o valor. Dica: `acervo[codigo][campo]` primeiro acha o livro e depois o campo.

5 · `listar_titulos(acervo)`	dificuldade: fácil
Recebe	o **acervo**.
Devolve	uma **lista** com o título de cada livro, na ordem do acervo (acervo vazio devolve lista vazia).
Regras	• Devolve os **títulos**, não os códigos.

Exemplos	`listar_titulos(acervo_exemplo)`
	`  devolve ['Dom Casmurro', 'Capitães da Areia', 'Vidas Secas', 'Quincas Borba']`
	`listar_titulos({})`
	`  devolve []`

Pense assim	1) Crie uma lista vazia. 2) Percorra o acervo; em cada livro, acrescente o título à lista. 3) No fim, devolva a lista.

Parte 2 — empréstimos e remoção · ≈ 60 min

Aqui as funções **alteram** os dicionários e algumas devolvem dois valores. As fichas 8, 9 e 10 trazem o passo a passo com as perguntas, na ordem em que devem ser feitas.

`6 · livros_do_aluno(emprestimos, aluno)`	**dificuldade: fácil**
Recebe	**emprestimos** (dicionário aluno → lista de códigos) e o nome do **aluno**.
Devolve	a **lista de códigos** que o aluno tem; **lista vazia** se ele não está em emprestimos.
Regras	• Não altera emprestimos. Em especial, **não crie** a chave de um aluno que não existe. • Esta função será reaproveitada por emprestar_livro e devolver_livro.

Exemplos	`livros_do_aluno(emprestimos_exemplo, "Ana")`
	`  devolve ['L001', 'L004']`
	`livros_do_aluno(emprestimos_exemplo, "Carla")`
	`  devolve []`
	`  depois: "Carla" in emprestimos_exemplo vale False`

Pense assim	1) O aluno está no dicionário? Então devolva a lista dele. 2) Se não está, devolva uma lista vazia.

`7 · livro_esta_emprestado(emprestimos, codigo)`	**dificuldade: média**
Recebe	**emprestimos** e o **codigo** de um livro.
Devolve	**True** se algum aluno está com o livro, **False** se ninguém está.
Regras	• Basta **um** aluno ter o código para a resposta ser True.

Exemplos	`livro_esta_emprestado(emprestimos_exemplo, "L002")`
	`  devolve True`
	`livro_esta_emprestado(emprestimos_exemplo, "L003")`
	`  devolve False`

Pense assim	1) Percorra todos os alunos de emprestimos. 2) Para cada aluno, o código está na lista dele? Se sim, já dá para responder True. 3) Só depois de olhar **todos** os alunos dá para responder False.
Atenção	devolver False logo no primeiro aluno que não tem o livro é um erro clássico: o livro pode estar com o segundo ou o terceiro.

8 · `emprestar_livro(acervo, emprestimos, aluno, codigo)`	dificuldade: média
Recebe	**acervo**, **emprestimos**, o **aluno** e o **codigo** do livro.
Devolve	**dois valores** de uma vez: `(ok, motivo)`, escritos assim: `return True, "ok"`.
Regras	• Recuse, verificando **nesta ordem**: "inexistente" (o código não está no acervo) → "sem exemplar" (exemplares == 0) → "limite" (o aluno já tem 3 livros). Recusa devolve (False, motivo) e **não altera nada**. • Se aceitar: exemplares do livro diminui 1; o código entra na lista do aluno em emprestimos; devolve (True, "ok").

Exemplos	`emprestar_livro(acervo_exemplo, emprestimos_exemplo, "Carla", "L004")`
	`  devolve (True, 'ok')`
	`  depois: acervo_exemplo["L004"]["exemplares"] vale 2`
	`  depois: emprestimos_exemplo["Carla"] vale ['L004']`
	`emprestar_livro(acervo_exemplo, emprestimos_exemplo, "Ana", "L003")`
	`  devolve (False, 'sem exemplar')`
	`emprestar_livro(acervo_exemplo, emprestimos_exemplo, "Ana", "L999")`
	`  devolve (False, 'inexistente')`
	`emprestar_livro(acervo_exemplo, emprestimos_exemplo, "Ana", "L002")  # Ana passa a ter 3 livros`
	`emprestar_livro(acervo_exemplo, emprestimos_exemplo, "Ana", "L001")`
	`  devolve (False, 'limite')`
	`  depois: emprestimos_exemplo["Ana"] vale ['L001', 'L004', 'L002']`

Pense assim	1) O código existe? Se não, recuse. 2) Ainda sobra exemplar? Se não, recuse. 3) Quantos livros o aluno já tem? (use livros_do_aluno) Se já tem 3, recuse. 4) Passou por todas as perguntas: agora sim faça as duas alterações. Se o aluno nunca pegou livro, ele ainda não existe em emprestimos: crie a lista dele e **guarde-a no dicionário** antes de acrescentar o código. 5) Devolva (True, "ok").
Atenção	se o aluno está no limite **e** o livro está sem exemplar, o motivo é “sem exemplar”: a ordem das perguntas manda. Alterar antes de ter certeza de que vai aceitar também é erro.

9 · devolver_livro(acervo, emprestimos, aluno, codigo)	dificuldade: média
Recebe	**acervo**, **emprestimos**, o **aluno** e o **codigo**.
Devolve	**dois valores**: `(ok, motivo)`.
Regras	• Recusa com (False, "nao emprestado") (sem acento) se o aluno não está com esse livro, **inclusive** se o aluno nem existe em emprestimos. • Se aceitar: o código sai da lista do aluno; exemplares aumenta 1; devolve (True, "ok"). • Se a lista do aluno ficar vazia, o aluno **deixa de existir** em emprestimos.

Exemplos	`devolver_livro(acervo_exemplo, emprestimos_exemplo, "Ana", "L001")`
	`  devolve (True, 'ok')`
	`  depois: emprestimos_exemplo["Ana"] vale ['L004']`
	`  depois: acervo_exemplo["L001"]["exemplares"] vale 3`
	`devolver_livro(acervo_exemplo, emprestimos_exemplo, "Bruno", "L002")`
	`  devolve (True, 'ok')`
	`  depois: "Bruno" in emprestimos_exemplo vale False`
	`devolver_livro(acervo_exemplo, emprestimos_exemplo, "Bruno", "L001")`
	`  devolve (False, 'nao emprestado')`
	`devolver_livro(acervo_exemplo, emprestimos_exemplo, "Carla", "L001")`
	`  devolve (False, 'nao emprestado')`
	`  depois: "Carla" in emprestimos_exemplo vale False`

Pense assim	1) Pegue a lista do aluno com livros_do_aluno (aluno que não existe já vem como lista vazia, sem erro). 2) O código está nessa lista? Se não, recuse. 3) Tire o código da lista e aumente 1 nos exemplares. 4) A lista ficou vazia? Então remova o aluno de emprestimos. 5) Devolva (True, "ok").
Atenção	uma lista vazia sobrando em emprestimos está errada: quem não tem livro não aparece no dicionário.

10 · remover_livro(acervo, emprestimos, codigo)	dificuldade: média
Recebe	**acervo**, **emprestimos** e o **codigo**.
Devolve	**dois valores**: `(ok, motivo)`.
Regras	• Recusa com (False, "inexistente") se o código não está no acervo. • Recusa com (False, "emprestado") se algum aluno está com o livro. • Senão apaga o livro do acervo e devolve (True, "ok").

Exemplos	`remover_livro(acervo_exemplo, emprestimos_exemplo, "L003")`
	`  devolve (True, 'ok')`
	`  depois: "L003" in acervo_exemplo vale False`
	`remover_livro(acervo_exemplo, emprestimos_exemplo, "L001")`
	`  devolve (False, 'emprestado')`
	`remover_livro(acervo_exemplo, emprestimos_exemplo, "L999")`
	`  devolve (False, 'inexistente')`

Pense assim	1) O código existe? Se não, recuse. 2) Alguém está com ele? (use livro_esta_emprestado) Se sim, recuse. 3) Apague a chave do acervo (cartão de referência) e devolva (True, "ok").

Treine no papel: sequência de chamadas

Dados de treino: `acervo = {"B1": ... exemplares 1, "B2": ... exemplares 2}` e `emprestimos = {}`. As chamadas acontecem **na ordem**, cada uma sobre o resultado da anterior. Cubra as três últimas colunas, preencha à mão e confira:

#	Chamada	Devolve	emprestimos depois	exemplares depois
1	`emprestar_livro(acervo, emprestimos, "Rui", "B1")`	`(True, 'ok')`	`{'Rui': ['B1']}`	`{'B1': 0, 'B2': 2}`
2	`emprestar_livro(acervo, emprestimos, "Eva", "B1")`	`(False, 'sem exemplar')`	`{'Rui': ['B1']}`	`{'B1': 0, 'B2': 2}`
3	`emprestar_livro(acervo, emprestimos, "Rui", "B9")`	`(False, 'inexistente')`	`{'Rui': ['B1']}`	`{'B1': 0, 'B2': 2}`
4	`devolver_livro(acervo, emprestimos, "Eva", "B1")`	`(False, 'nao emprestado')`	`{'Rui': ['B1']}`	`{'B1': 0, 'B2': 2}`
5	`devolver_livro(acervo, emprestimos, "Rui", "B1")`	`(True, 'ok')`	`{}`	`{'B1': 1, 'B2': 2}`
6	`remover_livro(acervo, emprestimos, "B1")`	`(True, 'ok')`	`{}`	`{'B2': 2}`
7	`emprestar_livro(acervo, emprestimos, "Eva", "B2")`	`(True, 'ok')`	`{'Eva': ['B2']}`	`{'B2': 1}`
8	`remover_livro(acervo, emprestimos, "B2")`	`(False, 'emprestado')`	`{'Eva': ['B2']}`	`{'B2': 1}`

Repare na chamada 4: Eva nunca teve livro, então a devolução é recusada *sem erro* e sem criar a Eva. Na 5, Rui devolve o único livro e **sai** de emprestimos.

Parte 3 — relatórios · ≈ 30 min

Três relatórios simples. Eles só **leem** os dados e devolvem o resultado; nenhum imprime.

`11 · relatorio_por_autor(acervo)`	dificuldade: média
Recebe	o **acervo**.
Devolve	um **dicionário**: autor → quantidade de títulos desse autor no acervo.
Regras	• Não altera o acervo. Acervo vazio devolve dicionário vazio.
Exemplos	`relatorio_por_autor(acervo_exemplo)` `  devolve {'Machado de Assis': 2, 'Jorge Amado': 1, 'Graciliano Ramos': 1}`
Pense assim	1) Comece com um dicionário vazio de contagens. 2) Para cada livro, leia o autor. 3) Primeira vez que o autor aparece: a contagem dele vira 1. Já apareceu antes: soma 1. (cartão: ler com padrão, `d.get(chave, 0)`)

`12 · relatorio_indisponiveis(acervo)`	dificuldade: fácil
Recebe	o **acervo**.
Devolve	uma **lista** com os **títulos** dos livros que estão com exemplares == 0, na ordem do acervo (lista vazia se não houver).
Regras	• Devolve os títulos, não os códigos.
Exemplos	`relatorio_indisponiveis(acervo_exemplo)` `  devolve ['Vidas Secas']`
Pense assim	1) Lista vazia. 2) Para cada livro, se exemplares é 0, acrescente o título. 3) Devolva a lista.

`13 · relatorio_emprestimos_mes(matriz, categorias, meses)`	dificuldade: difícil
Recebe	**matriz** (cada linha é uma categoria, cada coluna é um mês), a lista **categorias** e a lista **meses**. A linha i é a categoria `categorias[i]`; a coluna j é o mês `meses[j]`.
Devolve	**dois valores**: `(total_por_categoria, mes_mais_movimentado)`.
Regras	• **total_por_categoria**: dicionário categoria → soma da linha dela. • **mes_mais_movimentado**: o **nome** do mês (não a soma) com maior soma de coluna. Se houver empate, vale o primeiro. • Use len(categorias) e len(meses); a matriz de teste pode ter outro tamanho. Não altera a matriz.
Exemplos	`relatorio_emprestimos_mes(matriz_exemplo, categorias_exemplo, meses_exemplo)` `  devolve ({'Romance': 31, 'Didático': 27}, 'Fev')`
Veja	Para a matriz de exemplo, o resultado sai destas somas:

	Jan	Fev	Mar	soma da linha
Romance	10	12	9	31
Didático	4	15	8	27
soma da coluna	14	27	17	

Pense assim	1) Soma de uma **linha**: o índice i fica parado e o j varia. Zere o acumulador a cada linha nova. 2) Soma de uma **coluna**: o j fica parado e o i varia. Zere a cada coluna nova. 3) Para o mês: guarde a "melhor soma até agora" e o nome do mês dela; só troque se a nova soma for **maior** (por quê não "maior ou igual"?).

Parte 4 (desafio) — juntando tudo · ≈ 20 min

Uma só função que usa o que você construiu: é o sistema funcionando de ponta a ponta.

`14 · processar_movimentos(acervo, emprestimos, movimentos)`	**dificuldade: difícil**
Recebe	**acervo**, **emprestimos** e **movimentos**: lista de tuplas, na ordem em que aconteceram: `("emprestar", aluno, codigo)`, `("devolver", aluno, codigo)` ou `("remover", codigo)`.
Devolve	uma **lista de recusas**: tuplas `(numero_do_movimento, motivo)`, na ordem. O primeiro movimento é o número 1. Movimentos aceitos não entram na lista.
Regras	• Executa cada movimento chamando **emprestar_livro**, **devolver_livro** ou **remover_livro**, e aproveita o motivo que elas devolvem. • Os dicionários acervo e emprestimos ficam atualizados no final (as funções chamadas já alteram os dois).

Exemplos	`movs = [("emprestar", "Carla", "L004"), ("emprestar", "Carla", "L003"), ("devolver", "Bruno", "L001"),`
	`("remover", "L001"), ("remover", "L003"), ("emprestar", "Carla", "L003")]`
	`processar_movimentos(acervo_exemplo, emprestimos_exemplo, movs)`
	`devolve [(2, 'sem exemplar'), (3, 'nao emprestado'), (4, 'emprestado'), (6, 'inexistente')]`
	`depois: emprestimos_exemplo vale {'Ana': ['L001', 'L004'], 'Bruno': ['L002'], 'Carla': ['L004']}`
	`depois: list(acervo_exemplo.keys()) vale ['L001', 'L002', 'L004']`

Pense assim	1) Crie a lista de recusas e um contador que começa em 0 (o número não vem na tupla). 2) Para cada movimento: some 1 ao contador; olhe o primeiro item da tupla para saber qual função chamar. 3) A função chamada devolve (ok, motivo): guarde os dois em variáveis. Se ok for False, acrescente (número, motivo) às recusas. 4) No fim, devolva a lista de recusas.
Atenção	o movimento "remover" tem só 2 itens: o código está na posição 1 (nos outros dois está na posição 2). Esta função depende das funções 8, 9 e 10: se alguma delas ainda falha, confira-as primeiro.

Antes de entregar

1. Rodou `python testes.py` e leu as mensagens das funções com [X]? 2. As funções **devolvem** (return) em vez de imprimir? 3. Nenhuma função usa variável global nem valores fixos? 4. Respondeu R1 e R2 no final do arquivo? 5. Escreveu seu nome no topo e entregou o arquivo `solucao.py` até quinta-feira, 12/11, às 23h50?